Gymnázium, Praha 6, Arabská 14

ROČNÍKOVÁ PRÁCE

-

AUTOPŮJČOVNA

Obr. 1

Prohlášení

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne: _____

podpis: _____

Anotace

Tato práce se zabývá vytvořením jednoduché konzolové aplikace v javě. Program umožňuje evidenci aut, půjček i zákazníků. Uživatel může zobrazit nabídku aut, vyhledávat podle modelu nebo ceny, půjčit nebo vrátit auto a podobně. Při tvorbě projektu jsem využil základní principy objektově orientovaného programování.

Abstract

This work focuses on the creation of a simple car rental console application in java. The program allows the management of cars, rents and customers. The user can view the available cars, search by model or price, rent or return a car, etc. When creating the project, I used the basic principles of object oriented programming.

Zadání projektu

Cílem projektu bylo vytvořit program, který simuluje základní fungování autopůjčovny. Program měl umožňovat správu aut a zákazníků, půjčování a vracení aut, vyhledávání aut podle různých parametrů, výpočet ceny půjčky včetně slev a pokut a zobrazení základních statistik půjčovny.

Obsah:

1.Úvod
2.Popis projektu
2.1 Třída Main
2.2 Třída Pujcovna
2.2.1 Metody add
2.2.2 Seřazení objektů
2.2.3 Vyhledávání
2.2.4 Metoda filtrCena
2.2.5 Půjčování a vracení aut
2.3 Třídy Auto, Zakaznik a Pujcka
3.Závěr
4.Zdroje a obrázky

1. Úvod

Chtěl jsem vytvořit program který není moc složitý, procvičím si v něm základy objektově orientovaného programování, logiku programu a práci s daty, a zároveň mě bude bavit. Tak mě napadla autopůjčovna a proto jsem si ji zvolil jako téma ročníkové práce.

Celý program funguje hlavně díky třídám, objektům a metodám.
Při jeho tvorbě bylo potřeba řešit například:

- správu aut a zákazníků
- vyhledávání vozidel podle zadaných údajů
- evidenci aktivních půjček
- výpočet ceny půjčky včetně slev a pokut
- zobrazování statistik půjčovny

2. Popis projektu

2.1 Třída Main

Třída `Main` je hlavní vstupní bod celé aplikace. Jsou zde vytvořeny všechny počáteční objekty - tedy objekty aut, zákazníků, půjček, i půjčovna samotná. Tato třída obsahuje také hlavní menu, které je základem celé aplikace. Menu funguje na základě hodnoty proměnné int `vstupNum`, která je nastavena scannerem a přes podmínky typu if-else pak běží kód v podmínce, kde jsou volány metody z ostatních tříd.

Menu v terminálu:

```
Postupujte podle pokynů:

Pro výpis nabídky aut stiskněte 1+Enter
Pro vyhledávání auta stiskněte 2+Enter
Pro půjčení auta stiskněte 3+Enter
Pro vráceni auta stiskněte 4+Enter
Pro vyhledávání aut do určité ceny stiskněte 5+Enter
Pro statistiky půjčovny stiskněte 6+Enter
Pro ukončení programu stiskněte 0+Enter

Vaše volba:
```

Obr. 2

Příklad podmínky použité v menu:

```
if (vstupNum == 1) {
    pujcovna.serazeniCenaAVypis();
}
```

Obr. 3

2.2 Třída Pujcovna

Ve třídě `Pujcovna` funguje většina projektu. Zajišťuje hlavní operace systému půjčovny, jsou v ní založeny pole objektů a důležité metody, které celou autopůjčovnu pohání.

2.2.1 Metody add

`Pujcovna` obsahuje 3 metody *add* - a to `addAuto`, `addZakaznik` a `addPujcka`. Metody umožňují přidávat objekty aut, zákazníků a půjček do jejich polí.

Jedna z metod *add*:

```
public void addAuto(Auto auto) {
    auta[pocetAut++] = auto;
}
```

Obr. 4

2.2.2 Seřazení objektů

Metoda `serazeniCenaAVypis` v poli `auta[]` seřadí auta od nejnižší po nejvyšší cenu za den a na konec seřazené pole vypíše. Pro seřazení jsem využil algoritmus Bubble Sort.

```
public void serazeniCenaAVypis() {  1 usage  LukyK44
    for (int i = 0; i < pocetAut - 1; i++) {
        for (int j = 0; j < pocetAut - i - 1; j++) {
            if (auta[j].getCenaZaDen() > auta[j+1].getCenaZaDen()) {
                Auto temp = auta[j];
                auta[j] = auta[j+1];
                auta[j+1] = temp;
            }
        }
    }
    vypisAut();
}
```

Obr. 5

2.2.3 Vyhledávání

O vyhledávání modelu auta se ve třídě `Pujcovna` stará metoda `findAuto`. První for smyčka projede pole aut, vezme model a pomocí předdefinované metody `contains` zjistí, jestli obsahuje něco, co obsahuje i parametr `hledejModelAuta` (ten se díky scanneru fungujícím ve třídě `Main` nastavil na to, co uživatel zadal do konzole). Pokaždé když je nalezena shoda, `pocetShod` se o 1 zvýší. Následně zakládáme pole objektů `autaShody`, které je stejně velké jako dříve určená proměnná `pocetShod`. Teď jsem musel vyřešit jeden problém (vysvětlím později) a tak jsem si založil proměnnou `index`. Druhá for smyčka obsahuje stejnou if podmínku jako první, ale už opravdu dává objekty nalezených aut do našeho pole, které nakonec vracíme. Teď přichází na řadu `index`. Kdybych u pole `autaShody` totiž použil `i`, tak by se pole naplnilo špatně, protože by se do něj dal objekt na stejné místo jako v poli `auta`, některá místa v `autaShody` by tedy zůstaly prázdná a na některé objekty by nezbylo místo.

Stejnou metodu jsem napsal pro možné budoucí účely i pro objekty Zakaznik.

```
public Auto[] findAuto(String hledejModelAuta) {  1 usage  LukyK44
    int pocetShod = 0;
    for (int i = 0; i < pocetAut; i++) {
        if (auta[i].getModel().toLowerCase().contains(hledejModelAuta.toLowerCase())){
            pocetShod++;
        }
    }

    Auto[] autaShody = new Auto[pocetShod];
    int index = 0;
    for (int i = 0; i < pocetAut; i++) {
        if (auta[i].getModel().toLowerCase().contains(hledejModelAuta.toLowerCase())) {
            autaShody[index++] = auta[i];
        }
    }
    return autaShody;
}
```

Obr. 6

2.2.4 Metoda filtrCena

Chtěl jsem menu aplikace nějak zpestřit a napadla mě metoda, která umožní vyhledávat auta do určité denní ceny. Proto jsem ve třídě `Pujcovna` napsal metodu `filtrCena`. Metoda přijímá parametr `maxCena`, který je načten ze vstupu uživatele přes třídu `Main`. Základem metody je for smyčka, která porovnává jestli je denní cena aut nižší nebo stejně velká jako cena, kterou zadal uživatel.

```
public void filtrCena(int maxCena) {  1 usage  LukyK44 *
    boolean nalezeno = false;

    for (int i = 0; i < pocetAut; i++) {
        if (auta[i].getCenaZaDen() <= maxCena) {
            nalezeno = true;
            System.out.println(auta[i]);
        }
    }

    if (!nalezeno) {
        System.out.println("Nebyla nalezena žádná auta do požadované ceny za den.");
    }
}
```

Obr. 7

2.2.5 Půjčování a vracení aut

Půjčování a vracení aut nám umožňují metody `pujcitAuto` a `vratitAuto`.

Metoda `pujcitAuto` přijímá parametry `pocetDni`, objekt `auto` a objekt `zakaznik`. Celý kód metody leží v bloku try-catch. Nejdříve v bloku try připravíme výjimku na stav, kdy je auto už půjčené. Poté zavoláme všechny potřebné metody pro půjčení auta - vytvoříme novou půjčku, zvýšíme celkový počet půjček, zvýšíme počet půjček zákazníka a nakonec danou půjčku vrátíme. V bloku catch použijeme připravenou výjimku, vypíšeme zprávu a nakonec vrátíme půjčku nastavenou na null.

```
public Pujcka pujcitAuto(int pocetDni, Auto auto, Zakaznik zakaznik) {  1 usage

    try {
        if (!auto.getJeVolne()) {
            throw new IllegalStateException("Auto je momentálně vypůjčené");
        }

        auto.setJeVolne(false);
        auto.setTedPravePujcene(true);

        Pujcka pujckaX = new Pujcka(pocetDni, auto, zakaznik);

        pujcky[pocetPujcek++] = pujckaX;

        zakaznik.zvyseniPoctuZakPujcek();

        return pujckaX;

    } catch (IllegalStateException e) {
        System.out.println(e.getMessage());
        return null;
    }
}
```

Obr. 8

Metoda `vratitAuto` zase přijímá parametry `idAuta` a `faktDniPujcene`. Kód této metody se celý nachází ve for smyčce. Metoda nejdříve vezme id auta, které uživatel zadal. Poté uživatel zadá, jak dlouho auto skutečně měl. Na půjčku se zavolá metoda `pokuta`, která zjistí, jestli uživatel vrátil auto později, než na kolik dní si auto původně půjčoval. Pokud ano, bude mu přiúčtovaná pokuta 500 Kč za den navíc. Auto se následně nastaví na volné, vypíšou se půjčky, potom se sníží celkový počet půjček a daná půjčka se z pole půjček smaže.

```
public void vratitAuto(int idAuta, int faktDniPujcene) {  1 usage  LukyK44 *
        for (int i = 0; i < pocetPujcek; i++) {
            if (pujcky[i].getAuto().getId() == idAuta) {
                int pokuta = pujcky[i].pokuta(faktDniPujcene);
                if (pokuta > 0) {
                    System.out.println("Bylo Vám naúčtováno "
                     + pokuta + " Kč navíc za pozdní vrácení půjčenného auta.");
                }
                pujcky[i].getAuto().setJeVolne(true);
                vypisPujcek();

                pujcky[i] = pujcky[pocetPujcek - 1];
                pujcky[pocetPujcek - 1] = null;
                pocetPujcek --;
                vypisPujcek();
            }

        }
}
```

Obr. 9

2.3 Třídy Auto, Zakaznik a Pujcka

Třídy `Auto`, `Zakaznik` a `Pujcka` jsou všechny velmi podobné. Jejich hlavním úkolem je umožnit vytvářet objekty daných tříd. Každá třída má pro atributy konstruktor, gettery a settery. Některé nejsou využívány, ale chtěl jsem je mít vytvořené pro možný budoucí vývoj projektu. Pro výpis objektů využívám metodu `toString`.

Třída `Auto` má atributy: int `id`, String `znacka`, String `model`, boolean `jeManual`, int `rokVyroby`, int `cenaZaDen`, boolean `jeVolne` a pomocný boolean `tedPravePujcene`. Proměnnou `tedPravePujcene` jsem vytvořil čistě z kosmetického hlediska, protože kdyby existovala pouze proměnná `jeVolne`, tak to uživateli po půjčení auta vypíše, že auto už je půjčené, což by mohlo vyvolat dojem, že si ho půjčil jiný zákazník.

Třída `Zakaznik` má atributy: int `id`, String `jmeno`, String `prijmeni`, int `telCislo` a int `pujckyCount`. Tato třída má kromě prvků uvedených u těchto tří tříd navíc jen velmi jednoduchou metodu `zvyseniPoctuZakPujcek`, která zvýší `pujckyCount`, pokaždé, když je metoda zavolaná.

Třída `Pujcka` má atributy: int `pocetDni`, Auto `auto` a Zakaznik `zakaznik`. Obsahuje metodu `totalCena`, ta pomocí podmínek a jednoduchých výpočtů vytváří slevu celé půjčky, a metodu `pokuta`, která se stejným způsobem stará o přirážku, pokud zákazník vrátí auto později, než uvedl při půjčování auta.

3. Závěr

Díky tomuto projektu jsem si vyzkoušel, jak funguje tvorba programu, kde musí jednotlivé části spolupracovat a dávat smysl jako celek. Ukazuje také základní principy objektově orientovaného programování, jako je práce s třídami a objekty.

Projekt obsahuje několik tříd, které reprezentují reálné prvky systému autopůjčovny jako jsou například auta, zákazníci a půjčky. Tyto části společně vytvářejí funkční celek, který umožňuje správu dat a provádění základních operací. Program tak poměrně zajímavě simuluje reálnou situaci a zároveň ukazuje, kolik se toho člověk může naučit na ne moc velkém projektu.

Během práce jsem se naučil lépe organizovat kód, přemýšlet nad strukturou programu a řešit konkrétní problémy, které vznikaly při vývoji. Největší výzvou bylo zajistit, aby program fungoval spolehlivě ve všech situacích. Díky tomuto projektu jsem získal cenné zkušenosti, které mohu využít a také využiju při dalším programování.

4. Zdroje a obrázky

https://www.w3schools.com/java/java_classes.asp

https://www.w3schools.com/java/java_methods.asp

https://www.w3schools.com/java/java_try_catch.asp

https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/
https://www.w3schools.com/dsa/dsa_algo_bubblesort.php

Obr.1 - logo Gymnázia Arabská
Obr.2 - snímek obrazovky menu z konzole
Obr.3 - snímek obrazovky jedné ze sedmi podmínek využitých v menu
Obr.4 - snímek obrazovky metody addAuto
Obr.5 - snímek obrazovky metody serazeniCenaAVypis
Obr.6 - snímek obrazovky metody findAuto
Obr.7 - snímek obrazovky metody filtrCena
Obr.8 - snímek obrazovky metody pujcitAuto
Obr.9 - snímek obrazovky metody vratitAuto